

CUDA & GPU Programming Practice Test

Part 1 - CUDA Mini Projects, Tensor Cores & GPU Fundamentals — Questions 1-15 of 60 — Answers & Explanations

1. What does the `__global__` qualifier indicate when declaring a CUDA function in C/C++?

- A. The function is a global variable shared across all threads
- B. The function cannot accept any arguments
- C. The function only runs on the CPU
- D. The function is a kernel that runs on the GPU and is callable from the host

Correct Answer: D

Explanation: `__global__` marks a kernel function that executes on the device (GPU) but is launched from host (CPU) code.

2. What is the basic syntax used to launch a CUDA kernel with a specified grid and block configuration?

- A. `kernelName(gridDim, blockDim, args)`
- B. `launch kernelName gridDim blockDim args`
- C. `kernelName<<>>(args)`
- D. `kernelName::run(gridDim, blockDim, args)`

Correct Answer: C

Explanation: The triple angle bracket syntax `<<>>` specifies the execution configuration when launching a CUDA kernel.

3. What is the primary purpose of NVIDIA Tensor Cores?

- A. To replace the need for CUDA cores entirely
- B. To manage GPU memory allocation exclusively
- C. To accelerate mixed-precision matrix multiply-and-accumulate operations commonly used in AI workloads
- D. To increase the clock speed of the entire GPU

Correct Answer: C

Explanation: Tensor Cores are specialized hardware units designed to perform fast matrix multiply-accumulate operations, particularly benefiting deep learning workloads.

4. Why can using Tensor Cores significantly speed up deep learning training compared to standard CUDA cores?

- A. They replace the CPU entirely during training
- B. They only work with integer arithmetic
- C. They perform many matrix multiply-accumulate operations per clock cycle using mixed precision
- D. They eliminate the need for any GPU memory

Correct Answer: C

Explanation: Tensor Cores are purpose-built to execute matrix multiply-accumulate operations far more efficiently than general-purpose CUDA cores, especially in mixed precision.

5. What role does constant memory play in a CUDA image filter kernel?

- A. It stores data that changes every thread execution
- B. It replaces the need for global memory entirely

- C. It is only accessible from the host, not the device
- D. It stores read-only data, such as filter coefficients, that is cached and efficiently broadcast to all threads

Correct Answer: D

Explanation: Constant memory is optimized for data that doesn't change during kernel execution and is efficiently broadcast to threads that read the same value, such as filter kernels.

6. Why is GPU cache behavior important to consider when writing an image filter kernel?

- A. Efficient use of cache and memory access patterns can significantly affect kernel performance
- B. Cache behavior has no effect on GPU kernel performance
- C. GPUs do not have any cache
- D. Cache only affects CPU-side code, not GPU kernels

Correct Answer: A

Explanation: Just as with CPUs, how a GPU kernel accesses memory affects cache hit rates, which can substantially impact overall performance.

7. In a simple CUDA matrix multiplication kernel, what typically determines which output element a thread computes?

- A. The total amount of GPU memory available
- B. The thread's global index, derived from its block and thread indices
- C. The order in which threads are declared in the source code
- D. A random number generated at runtime

Correct Answer: B

Explanation: Each thread typically computes its row and column position from blockIdx, blockDim, and threadIdx to determine which output element it is responsible for.

8. What is a Streaming Multiprocessor (SM) in NVIDIA GPU architecture?

- A. A software tool for compiling CUDA code
- B. The main system memory of the computer
- C. A type of network interface for multi-GPU communication
- D. A hardware unit that manages and executes groups of threads (warps) using its own cores and resources

Correct Answer: D

Explanation: The SM is a core hardware building block of the GPU responsible for scheduling and executing warps of threads with dedicated compute and memory resources.

9. Why is understanding GPU hardware architecture important for CUDA C programmers?

- A. It has no impact on how kernels should be written
- B. It determines which programming language must be used
- C. It helps programmers write kernels that better utilize parallelism and memory bandwidth
- D. It is only relevant for GPU driver developers, not application programmers

Correct Answer: C

Explanation: Knowledge of the underlying hardware helps programmers design kernels that align with how the GPU actually schedules threads and accesses memory, improving performance.

10. What is memory coalescing in the context of CUDA programming?

- A. Combining multiple threads' memory accesses into fewer, larger transactions when they access contiguous memory

- B. Reducing the number of threads used in a kernel
- C. Compressing GPU memory to save space
- D. Merging multiple kernels into a single kernel launch

Correct Answer: A

Explanation: Memory coalescing occurs when threads in a warp access contiguous memory locations, allowing the hardware to combine those accesses into fewer, more efficient transactions.

11. Why can changing just a couple of variables in a CUDA kernel sometimes yield a dramatic speedup like 4.5x, as with improved memory coalescing?

- A. It only works when the kernel uses no memory at all
- B. Variable names alone directly affect GPU clock speed
- C. Small changes to memory access patterns can transform inefficient scattered accesses into coalesced ones
- D. The GPU automatically rewrites the kernel regardless of code changes

Correct Answer: C

Explanation: Because coalescing depends heavily on access patterns, small adjustments—like reordering indices—can shift memory access from scattered to contiguous, dramatically improving throughput.

12. What is the main idea behind tiled matrix multiplication in CUDA C?

- A. Splitting the computation across multiple separate GPUs only
- B. Dividing matrices into smaller tiles loaded into fast shared memory to reduce redundant global memory accesses
- C. Performing the entire multiplication using only registers
- D. Avoiding the use of threads altogether

Correct Answer: B

Explanation: Tiling loads sub-blocks (tiles) of the input matrices into shared memory, allowing threads in a block to reuse that data and reduce costly repeated global memory reads.

13. Why is tiled matrix multiplication considered a 'must-know' GPU technique?

- A. It eliminates the need for a GPU altogether
- B. It is required syntax for any CUDA kernel to compile
- C. It only works for very small matrices under 4x4
- D. It significantly improves performance by better utilizing shared memory and reducing global memory traffic

Correct Answer: D

Explanation: Because it substantially reduces global memory bandwidth usage while increasing data reuse, tiling is a foundational optimization technique in GPU matrix computations.

14. What is CuPy primarily used for in Python-based GPU programming?

- A. Replacing the need for any GPU hardware
- B. Managing Python package installations
- C. Providing a NumPy-compatible array interface that runs computations on the GPU
- D. Compiling C++ code into Python bytecode

Correct Answer: C

Explanation: CuPy offers an array library with an API closely mirroring NumPy, but executes operations on the GPU using CUDA under the hood.

15. What is a key benefit of using Numba for CUDA programming in Python?

- A. It only supports CPU execution, not GPU
- B. It requires rewriting all code in C++ first
- C. It allows writing custom GPU kernels directly in Python syntax, which are then JIT-compiled for the GPU
- D. It automatically converts Python code into a mobile app

Correct Answer: C

Explanation: Numba's CUDA JIT compiler lets developers write GPU kernels using Python syntax, which are compiled just-in-time to run on NVIDIA GPUs.